{EPITECH}

AREA_

< BOOTSTRAP - CHOOSE. PLAN. SHIP. />



AREA

How to use this bootstrap

This is a sandbox, not a syllabus. The exercises below are offered to you, not imposed on you: pick the ones that match what your team actually needs to work on, drop the rest, and expect to fail a few times inside them rather than during the project. Each exercise is sized for 40 to 90 minutes and can be run independently of the others.

Whatever else you skip, do **at least one exercise from each of the two sections below**. Teams that only work on the stack arrive with three prototypes and no schedule; teams that only work on the planning arrive with a beautiful schedule for a stack they cannot use. Both fail for the same reason, and both fail late.

Every exercise asks you to produce a file and commit it. That is deliberate: a decision you cannot show is a decision you did not make.

You definitely should explore the whole document. But if time is tight, prioritize these MUST-DOs:

- ✓ Three timeboxed spikes, because you cannot compare technologies you have never run.
- ✓ Thirty tasks to move out, because a task list without dependencies is a wish list.

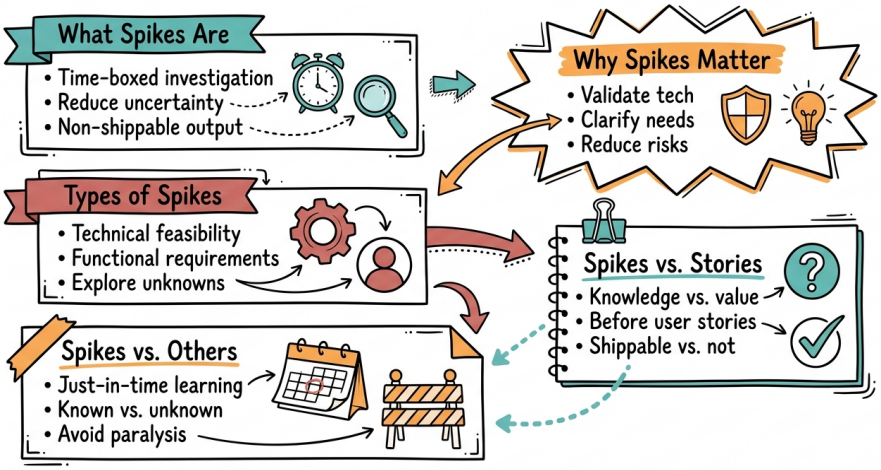
Everything else in this document deepens one of those two moves. Add exercises from there according to where your team is weakest, not according to what looks quickest.

Stack Attack

Objective: be able to select a language, a framework and a database for a real constraint set, and defend that selection with evidence you produced yourself rather than with what you read on a comparison blog.

Three timeboxed spikes

A spike is a fixed slice of time spent buying information, not building a feature. It comes from Extreme Programming and the rule that makes it work is brutal: when the timer rings, you stop, working or not.



Pick three backend technologies that nobody in your team has already shipped something with. Agree on a single timebox per technology and write down why you chose that length. Then, inside each timebox, reach the same target in all three:

- ✓ a server listening on port `8080`
- ✓ answering `GET /ping` with the exact body `{"status":"ok"}`
- ✓ reading the port from an environment variable rather than a literal in the source

Check yourself with `curl http://localhost:8080/ping`.

When the timer rings, stop. Immediately write `spikes/<technology>/SPIKE.md` recording, at minimum: where you were when time ran out, what consumed the most minutes, and what you would need to go further.

Then write `spikes/DISTANCE.md`, a table comparing the three runs. You decide what ”distance travelled” means here and you justify that definition: it is the whole point of the exercise.



Resist the urge to extend the timebox on the technology you already like. That reflex is exactly the bias this exercise is meant to expose.

The weighted decision matrix

Run this one after the spikes. Filled in before you have touched anything, a matrix produces invented scores; filled in after, it produces defensible ones.

Define and lock your criteria first: 5 to 8 criteria that genuinely matter for this project and this team. Then assign each criterion a weight from 1 to 3, and agree on what those weights mean before anyone scores anything.

Each team member now fills the matrix **independently, without discussing with anyone**, scoring every option from 1 to 5 per criterion, then computing the weighted total.
Here is an example of what it COULD look like.

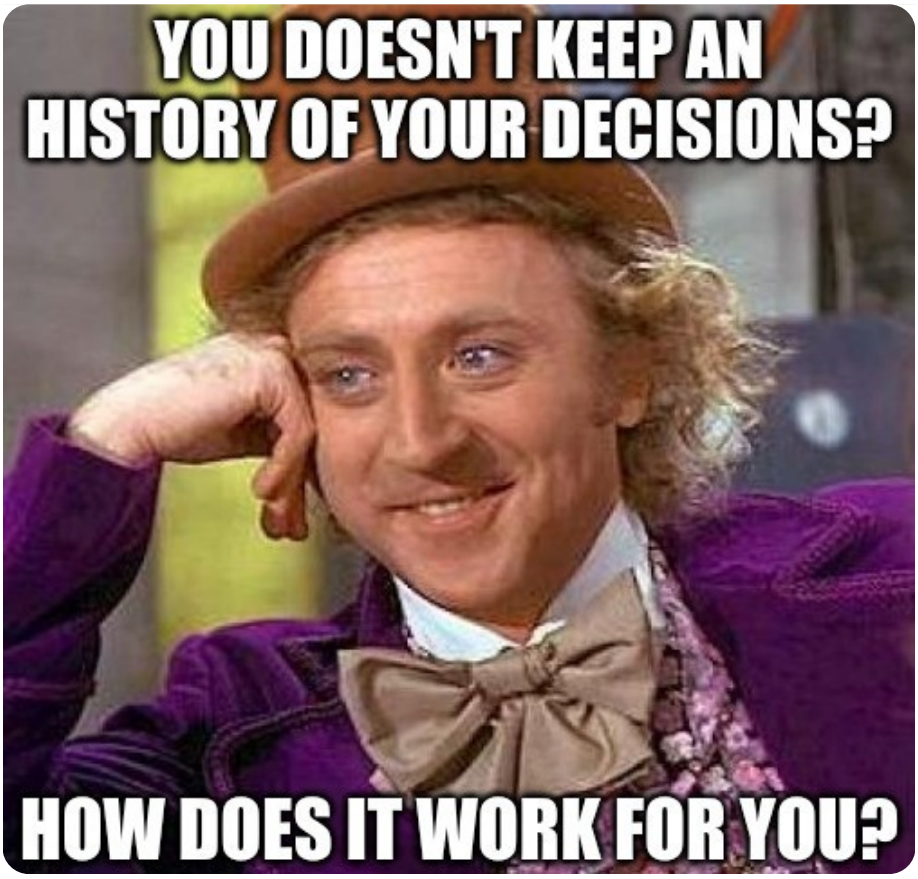
Weighted Decision Matrix - Software Development Project 1							
		OPTIONS					
Criteria	Weighting	Solution A		Solution B		Solution C	
		Score	Total	Score	Total	Score	Total
Infra Cost	3	5	15	3	9	4	12
Development Time	3	3	9	5	15	2	6
System Security	5	2	10	5	25	5	25
Ability to simulate the end to end flows and write test programs	4	4	16	4	16	3	12
External audit cost	3	4	12	5	15	3	9
	TOTAL:		62		80		64

Only then compare matrices. Flag every criterion where two members diverge by more than 2 points and treat each divergence as a finding, not as a rounding error: it means the two of you are not evaluating the same thing.

Close the exercise with `decision/RECOMMENDATION.md`, 250 to 300 words, covering your chosen stack, the two criteria that actually drove the decision, one trade-off you are knowingly accepting and how you intend to survive it, and one disagreement inside the team and how it was settled.

Architecture decision records

An **ADR** captures one decision, its context and its consequences, and lives in the repository next to the code. It is the format used to keep architecture decisions auditable long after the people who made them have moved on, and it is the artefact that turns ”we picked Postgres” into something a newcomer can evaluate.



Write one ADR per stack decision you have made, as `docs/adr/0001-<slug>.md`, `docs/adr/0002-<slug>.md` and so on. Find the field structure in the resources below rather than inventing one. Two constraints are non negotiable:

- ✓ each record states its status explicitly
- ✓ each record names at least one negative consequence you are accepting

An ADR with no cost attached to it is a press release, not a decision record.

Self check: hand one ADR to someone from another team who was not in the room. They must be able to restate your decision and its cost without asking you a single question. If they cannot, the record is incomplete.

Other selection methods worth knowing

Weighted matrices, spikes and ADRs are three tools among many. If you want to go further, or if none of the three fits how your team argues, these are the names professionals actually use:

- ✓ Technology Radar and build your own radar, mapping technologies into adopt, trial, assess and hold rings
- ✓ Choose Boring Technology and its innovation tokens, budgeting how much novelty a team can afford
- ✓ Architecture Tradeoff Analysis Method (ATAM), evaluating an architecture against measurable quality scenarios and naming its tradeoff points
- ✓ Trade study, the systems engineering form of a documented comparison between alternatives
- ✓ Analytic hierarchy process (AHP), a pairwise comparison method for deriving weights instead of guessing them
- ✓ Delphi method, converging a group of experts anonymously to avoid the loudest voice winning
- ✓ Wardley map, positioning components by evolution to decide what to build and what to consume
- ✓ Total cost of ownership, the build versus buy argument counted over the whole life of the thing
- ✓ Request for Comments (RFC), the written proposal circulated for review before a decision is taken
- ✓ Proof of concept, and its close relative the bake-off, where every candidate ships the same feature and only measurements count

Keep Calm And Carry On

Objective: be able to break work into tasks that can be scheduled, spot which tasks drag everything else with them, and defend a scope decision under a constraint you cannot negotiate away.

Project management is not a school subject. You already run projects: a trip, a move, a party, a shared flat. The three exercises below start there on purpose, because the mechanics transfer, and because it is much easier to see a planning mistake when the cost is a wasted Saturday rather than an abstract deadline.

Thirty tasks to move out

You are moving out of your studio in three weeks. Hand over the keys on the wrong day and you pay another month of rent.



Break the move into tasks of half a day at most. Aim for something like thirty of them: give notice, book the van, find boxes, pack, final inspection, forward your mail, move the internet contract, get the deposit back, clean. Then, for each task, record its duration and its predecessors, in `planning/moving-wbs.md`:

```
| Task | Duration | Predecessors |
| ---- | -------- | ------------ |
```

Now find the longest chain of dependent tasks from start to finish and highlight it. Answer these in the same file:

- ✓ which task can slip by two days without moving your key handover date
- ✓ which task moves everything the moment it slips by one hour
- ✓ where you have two tasks fighting over the same person on the same day

Then do it again, in `planning/project-wbs.md`, on the weeks between today and your own first deadline. Same columns, same highlighted chain. Compare the two: the second one is always the one people get wrong, and it is rarely because the project is harder.

A party on a fixed budget

Forty people, a date that cannot move, a budget that cannot grow. Every party you have been to was a scope negotiation, whether anyone said so or not.



List everything you would want: venue, sound, food, drinks, decoration, someone on the door, cleaning, a plan for the neighbours. Then sort all of it into four buckets: must have, should have, could have, and won't have this time.

Two rules make this exercise work:

- ✓ the `won't have this time` bucket cannot be empty
- ✓ everything you drop into it has to be said out loud to the group, not silently deleted

Save that as `planning/moscow-v1.md`. Then cut your budget by thirty percent and redo the whole sort into `planning/moscow-v2.md`. Note in the second file which items moved and what that movement tells you about your first pass.

Finish by applying the same four buckets to your own feature list. You choose how much of your effort each bucket is allowed to consume, and you justify that split: a plan where everything is a must have is a plan with no room to absorb a single bad week.

{EPITECH}

The flatshare kanban

Four people share a flat for a week.



Model that week on a board with four columns: to do, doing, blocked, done. Groceries, bins, the broken shower, the landlord who never answers, the friend sleeping on the couch on Thursday.

Run it once with no constraint. Then impose a limit on how many items each column may hold at any time, and live with it: when a column is full, nobody may pull anything new, so the only way forward is to unblock what is already there.

You choose the limits and you justify them. Answer, in writing:

- ✓ what limit did you set per column, and on what basis
- ✓ what became visible once the limit was in place that was invisible before
- ✓ who stopped being able to start things, and whether that was a problem or the point

Save the board before and after as `planning/board-before.md` and `planning/board-after.md`, then set up your own project board with your own limits and commit it as `planning/board-project.md`.



The uncomfortable part of this exercise is the `blocked` column filling up while nobody is allowed to start anything new. That discomfort is information: it is the same discomfort your project will produce in week three, except there it will be silent.

Other project management methods to explore

The three exercises above touch task breakdown, prioritisation and flow. There is a great deal more, and knowing the vocabulary is what lets you pick deliberately instead of defaulting to whatever tool your group already had an account for:

- ✓ Work breakdown structure and critical path method, the formal versions of the first exercise
- ✓ Critical path, float and slack, how much a task can slip before it costs you
- ✓ Gantt chart and PERT, the two classical ways of drawing a schedule
- ✓ MoSCoW method, the prioritisation framework behind the second exercise
- ✓ Kanban and work in progress limits, the practice behind the third
- ✓ Scrum, Extreme Programming and SAFe, three very different answers to how a team organises itself
- ✓ PRINCE2 and the V-model, what project management looks like when the plan comes first
- ✓ Project premortem and pre-mortem, imagining the failure before it happens to find risks you would otherwise miss
- ✓ Planning fallacy and reference class forecasting, why your estimates are wrong and what actually fixes them
- ✓ Planning poker and burndown chart, estimating together and watching whether the estimate holds
- ✓ Responsibility assignment matrix (RACI), making it explicit who decides and who is merely consulted
- ✓ Kano model and OKR, two other ways of deciding what is worth doing
- ✓ Retrospective, the habit that turns one bad week into something the team does not repeat



v 1.1.1

